MINISTRY OF EDUCATION AND TRAINING
NATIONAL ACADEMIC RESEARCH INSTITUTE
FACULTY OF INFORMATION TECHNOLOGY AND CYBERSECURITY

***

ACADEMIC CAPSTONE RESEARCH REPORT
CRYPTOGRAPHY AND INFORMATION ASSURANCE

REAL-WORLD APPLICATIONS OF RSA: CRYPTOGRAPHIC FOUNDATIONS, PRACTICAL IMPLEMENTATION, AND OFF-CHAIN BLOCKCHAIN AUTHENTICATION

RESEARCH PROJECT MEMBER LIST / STUDENT IDENTIFICATION

No.	Full Name	Student ID
1	.....	.....
2	.....	.....
3	.....	.....
4	.....	.....
5	.....	.....
6	.....	.....
7	.....	.....

HANOI, 2026

Real-World Applications of RSA: Cryptographic Foundations, Practical Implementation, and Off-Chain Blockchain Authentication

Academic Research Paper in Cryptography, Cybersecurity, and Blockchain Systems

Abstract— The RSA algorithm remains a foundational pillar of modern public-key cryptography. While public blockchain consensus protocols predominantly utilize elliptic-curve cryptography (ECC) for on-chain state compactness, RSA plays an enduring role in enterprise infrastructure, Public Key Infrastructure (PKI), and internal gateway authorization. This report provides an academic investigation of real-world RSA applications across theory, operational justification, and systems engineering. We formulate RSA's mathematical foundations, contrasting RSA-OAEP key transport with RSA-PSS digital signatures (WHAT). We analyze why RSA is favored in specific trust architectures, providing an evidence-based comparison against ECDSA and Schnorr signatures (WHY). Finally, we design, implement, and benchmark an educational off-chain cryptocurrency withdrawal authentication system using RSA-PSS and SHA-256 (HOW). Empirical benchmarks across RSA (2048, 3072, 4096 bits) and ECDSA (secp256k1) on identical hardware demonstrate RSA's distinct operational profile: while RSA signing is slower than ECDSA, RSA public exponent verification (e = 65,537) executes significantly faster across all tested key sizes—RSA-2048 is 11.32x faster, RSA-3072 is 6.18x faster, and RSA-4096 is 4.17x faster than ECDSA verification. At the approximately equivalent 128-bit classical security level (RSA-3072 vs. ECDSA-secp256k1), RSA-3072 verification is 6.18x faster than ECDSA, whereas ECDSA signing is 5.82x faster than RSA-3072. Adversarial security scenario testing demonstrates that combining cryptographic digital signatures with application-level timestamp freshness and nonce replay caches significantly strengthens request authentication against payload tampering and replay attacks. Evaluation against the bundled RSA-PSS 2048-bit SHA-256/MGF1-SHA256 Wycheproof test-vector set (version 0.9 snapshot, 108 cases) confirms that the verification engine correctly accepts all 63 expected-valid test cases and rejects all 45 expected-invalid test cases (0 skipped, 0 errors), validating execution against this selected verification suite.

Keywords: *RSA Cryptography, RSA-PSS, Digital Signatures, PKI, Hybrid Encryption, Cryptocurrency Security, Off-chain Authentication, Empirical Evaluation.*

1. Introduction and Research Scope

First documented by Ron Rivest, Adi Shamir, and Leonard Adleman in their April 1977 MIT research memorandum and published in Communications of the ACM in 1978 [1], and formally standardized under PKCS #1 / RFC 8017 [2], the RSA algorithm solved the historic key distribution problem by separating the private transformation used for decryption and signing from the public transformation used for encryption and verification. For nearly five decades, RSA has underpinned digital commerce, software distribution, and enterprise identity management.

In modern blockchain environments, significant architectural debate centers on algorithm selection. Major public networks—including Bitcoin and Ethereum—rely on elliptic-curve cryptography (specifically ECDSA and Schnorr signatures over secp256k1) due to compact key and signature sizes that minimize consensus ledger bloat. Consequently, a misconception has emerged that RSA is obsolete. This report counters that assumption by demonstrating that cryptographic utility is context-dependent. While ECC is optimal for resource-constrained consensus networks, RSA provides unmatched public-key verification throughput and mature Hardware Security Module (HSM) support, making it exceptionally well-suited for high-volume off-chain authorization pipelines.

2. RSA Fundamentals and Real-World Applications

2.1 Mathematical Foundations and Operating Mechanics

The security of RSA rests on the integer factorization problem—the computational difficulty of determining the prime factors p and q of a large composite modulus n = p · q. Key generation selects two distinct large prime numbers p and q. The modulus n and Euler's totient function φ(n) = (p − 1)(q − 1) are computed. A public exponent e is chosen such that $1 < e < \varphi(n)$ and gcd(e, φ(n)) = 1 (normatively standardized as e = 65,537 = $2^{16} + 1$). The private exponent d is computed as the modular multiplicative inverse: $d \equiv e^{-1} \pmod{\varphi(n)}$, satisfying $e \cdot d \equiv 1 \pmod{\varphi(n)}$. The public key consists of (n, e), and the private key comprises (n, d) or Chinese Remainder Theorem (CRT) representations (p, q, d_p, d_q, qInv). Mathematically, Euler's totient theorem guarantees that for any message integer m coprime to n, (m^e)^d ≡ m^(e·d) ≡ m^(k·φ(n)+1) ≡ (m^(φ(n)))^k · m ≡ 1^k · m ≡ m (mod n). RSA forms a trapdoor one-way permutation: computing x^e mod n is computationally trivial given public (n, e), but computing the inverse x = y^d mod n without private trapdoor d is computationally intractable.

2.2 Distinction Between RSA Encryption and RSA Digital Signatures

RSA supports two distinct operational primitives designed for contrasting security goals:

• Asymmetric Encryption / Key Transport (RSA-OAEP): Provides confidentiality. The sender encrypts plaintext M using public key (n, e): C = RSAEP((n, e), M). Plaintext length is strictly bounded: mLen ≤ k − 2hLen − 2 (for RSA-2048 with SHA-256, mLen ≤ 190 bytes). Optimal Asymmetric Encryption Padding (OAEP) provides semantic security (IND-CCA2 in the Random Oracle Model under standard RSA one-wayness via Fujisaki et al. [14], resolving Shoup's analysis [13] of Bellare-Rogaway [15]).

• Digital Signatures (RSA-PSS): Provides entity authentication, payload integrity, and non-repudiation. The signer signs message M using private key d: computes digest mHash = Hash(M), generates encoded octet string EM = EMSA-PSS-ENCODE(mHash, emBits, sLen) with randomized salt, converts to integer representative m = OS2IP(EM), and computes signature s = RSASP1(SK, m) = m^d mod n (RFC 8017 Section 8.1 [2]). The verifier recovers m = s^e mod n and verifies EMSA-PSS padding consistency. RSA-PSS achieves tight existential unforgeability under adaptive chosen-message attack (EUF-CMA) in the ROM (Bellare-Rogaway [16]). Normative standard FIPS 186-5 [4] strictly mandates key separation: an RSA key pair designated for digital signatures shall never be used for encryption.

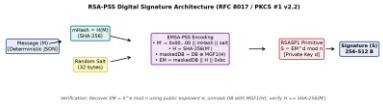


Figure 1. Operational workflow of the RSA-PSS probabilistic digital signature scheme (RFC 8017 / FIPS 186-5).

2.3 Real-World Application Breadth

In industrial production, RSA is deployed across three primary operational domains: (1) Hybrid Encryption and Session Key Transport: In TLS 1.2 and secure protocols, RSA-OAEP encrypts ephemeral symmetric session keys (AES-256-GCM), combining asymmetric key exchange with high-throughput bulk cipher streams. (2) Transport Layer Security (TLS 1.3) Endpoint Authentication: Under RFC 9846 [3], static RSA key exchange was completely deprecated to mandate Forward Secrecy via ephemeral Diffie-Hellman ((EC)DHE). However, RSA remains fully supported and widely deployed for server CertificateVerify endpoint authentication, with RSA-PSS explicitly required for modern TLS 1.3 certificate validation. (3) Enterprise Public Key Infrastructure (PKI) and Code Signing: X.509 Certificate Authorities (CAs) and OS software validation pipelines leverage RSA-3072/4096 root certificates due to universal enterprise interoperability.

3. Why RSA? Security Purpose and Comparative Evaluation

Table 1 systematically compares RSA configurations with modern elliptic curve alternatives: ECDSA (secp256k1) [11], Schnorr (BIP 340) [7], and Ed25519 [10]. Security strengths are classified according to NIST SP 800-56B Rev. 2 [5] and NIST SP 800-57 Part 1 Rev. 5 [6].

Evaluation Metric	RSA-2048	RSA-3072	ECDSA (secp256k1)	Schnorr (BIP 340)	Ed25519 (RFC 8032)
Hard Problem	Factoring	Factoring	EC Discrete Log	EC Discrete Log	Edwards EC-DLOG
Security Strength	112 bits [5]	128 bits [5]	~128 bits [6]	128 bits [6]	128 bits [6]
Key Size / Sig Size	Modulus: 256 B Sig: 256 B	Modulus: 384 B Sig: 384 B	PubKey: 33 B (comp) Sig: 70–72 B (DER)	PubKey: 32 B (x-only) Sig: 64 B (BIP341)	PubKey: 32 B Sig: 64 B [10]
Relative Verify Speed	Empirical: 0.021 ms (11.32x vs ECDSA)	Empirical: 0.039 ms (6.18x vs ECDSA)	Empirical: 0.242 ms (Baseline: 1.0x)	Batch verify supported; not benchmarked [7]	High efficiency in lit. [10]; not benchmarked

Table 1. Systematic comparison of asymmetric cryptographic algorithms across security and operational metrics.

3.1 Storage and Verification Trade-Offs in Blockchain Consensus Layers

A central requirement of public blockchains is minimizing ledger bloat and bandwidth overhead. In Bitcoin, every full node must validate and store all consensus signatures. A standard SegWit P2WPKH witness stack totals approximately 107–109 bytes (~70–72B DER signature + 33B compressed pubkey) [8, 11]. In a hypothetical architecture where RSA-2048 were used for native transaction authorization, carrying two stack items (256B modulus and 256B signature, each with 3B length prefix) yields 1 + 3 + 256 + 3 + 256 = 519 bytes, adding 410–412 witness bytes per input (~103 additional virtual bytes under BIP 141 weight rules [8]). For a standard 141-vB transaction, this represents a ~73% expansion to ~244 vB, directly inflating miner fees. Native Bitcoin script also lacks RSA opcodes. Similarly, Ethereum consensus rules mandate ECDSA over secp256k1 for native Externally Owned Account (EOA) envelopes [12]. While smart contracts can execute raw modular exponentiation via EVM precompile 0x05 (EIP-198), it computes B^E mod M only; executing full RSA-PSS unmasking, MGF1 hashing, and padding checks requires substantial EVM bytecode gas overhead, making consensus-layer RSA economically impractical. In contrast, BIP 340 Schnorr signatures are exactly 64 bytes and enable two-round multi-signature aggregation via MuSig2 (BIP 327 [9]), compacting n-of-n signers into a single 64-byte signature—a capability unattainable with standard RSA.

3.2 The Operational Value of RSA in Off-Chain Systems

Despite on-chain limitations, RSA provides decisive operational advantages in off-chain exchange authorization backends: (1) Asymmetric Verification Speed: RSA public exponentiation with $e = 65{,}537 = 2^{16} + 1$ (Hamming weight 2) executes in only 16 modular squarings and 1 modular multiplication (17 operations total), enabling ultra-rapid edge verification. (2) Enterprise HSM Support: Commercial banking HSMs provide certified hardware support for RSA modular arithmetic. (3) Resistance to Scalar Nonce Reuse: Standard ECDSA is catastrophically fragile: if signing nonce k repeats or exhibits slight bit bias, private key d_A is algebraically exposed. RSA-PSS eliminates scalar nonce exposure. However, implementations must employ base blinding ($x' = x \cdot r^e \bmod n$) against timing attacks. Crucially, blinding does not mitigate Chinese Remainder Theorem (CRT) hardware fault injection attacks (Bellcore attack; Boneh et al. [7]), where corrupted execution yields $\gcd(s - s', n) \in \{p, q\}$. Mitigating CRT fault injection strictly requires post-computation verification (verifying $s^e \equiv m \bmod n$ prior to signature release) or redundant fault-detection hardware.

4. Proposed System Design and Implementation

We design and evaluate an educational, off-chain architecture: Using RSA-PSS Digital Signatures to Authenticate Off-Chain Withdrawal Requests in Cryptocurrency Exchanges. Important Technical Boundary: The proposed architecture operates strictly off-chain within an internal authorization boundary to authenticate withdrawal requests before they reach hot-wallet signers; it does not replace on-chain ECDSA/Schnorr consensus signatures on Bitcoin or Ethereum, nor does it represent proprietary commercial exchange systems.

4.1 Architecture and Component Decomposition

As illustrated in Figure 2, the pipeline decouples request origination from on-chain broadcast across four functional tiers: 1. Withdrawal Service: Formats request parameters: network, recipient address, exact atomic integer amount (wei), UUIDv4 nonce, and UNIX timestamp. 2. RSA Signing Component: Encapsulated within a secure enclave/HSM; formats deterministic JSON, computes SHA-256 digest, and signs via RSA-PSS. 3. Verification Gateway: Edge service that retrieves exchange RSA public key, reconstructs message digest, and validates PSS signature. 4. Request Validation Engine: Enforces timestamp freshness (±300s window) and checks nonce uniqueness against an anti-replay store before dispatching to the on-chain Hot Wallet.



Figure 2. Proposed off-chain cryptocurrency withdrawal authentication architecture using RSA-PSS.

4.2 Deterministic JSON Serialization and Pipeline Algorithm

Cryptographic verification fails if microservices serialize JSON keys or whitespace inconsistently. Our prototype implements deterministic serialization by sorting dictionary keys lexicographically, eliminating whitespace, and encoding to UTF-8. (Note: While providing byte-level determinism for the schema, this does not implement full RFC 8785 JCS number formatting). Per RFC 8017 Section 9.1 [2], EMSA-PSS encoding specifies emBits = modBits − 1 (2047 bits for RSA-2048, emLen = 256 bytes; salt sLen = 32 bytes). Algorithm 1 outlines the complete pipeline.

```
Algorithm 1: Off-Chain Withdrawal Authentication Pipeline
Input:  Request R = {id, net, addr, amt_atomic, nonce, ts}, Private Key SK_rsa, Public Key PK_rsa
Output: Decision ∈ {ACCEPT, REJECT_SIG, REJECT_REPLAY, REJECT_EXPIRED}
1:  M_canon ← DeterministicJSON(R)                      // Lexicographical keys, compact separators, UTF-8
2:  EM ← EMSA-PSS-ENCODE(M_canon, emBits=modBits-1, sLen=32)  // RFC 8017 PSS encoding
3:  σ ← RSASP1(SK_rsa, OS2IP(EM))                       // Signature generation
4:  // Gateway Verification Phase
5:  valid_sig ← RSASSA-PSS-Verify(PK_rsa, M_canon, σ, SHA-256, sLen=32)
6:  if ¬ valid_sig then return REJECT_SIG                 // Tampering / invalid key detected
7:  if |CurrentTime() − R.ts| > Δ_skew then return REJECT_EXPIRED
8:  if R.nonce ∈ NonceStore then return REJECT_REPLAY      // Replay detected
9:  NonceStore.Add(R.nonce); return ACCEPT
```

4.3 Prototype Scope and Vital Security Considerations

Academic integrity requires explicit delineation between prototype capabilities and production requirements: (1) Stateful Replay Prevention: The prototype verifies and stores nonces in an in-memory Python set. In distributed production environments, nonces must be stored atomically (e.g., Redis SETNX or SQL unique constraint) with TTL eviction to prevent race conditions and cross-worker replays. (2) Address Validation: Prototype applies sanity checks (0x prefix, length); production requires strict EIP-55 (Ethereum) or Bech32 (Bitcoin) checksum validation. (3) Key Management: Signing keys reside in memory for demonstration; production mandates FIPS 140-3 validated HSMs, key rotation, and CRL/OCSP revocation. (4) Authorization vs Identity: A cryptographically valid RSA signature proves possession of the corresponding private key and technical payload integrity; it does not independently verify end-user legal identity, beneficial account ownership, KYC status, or authorization to withdraw funds, which must be enforced by business compliance tiers.

5. Experimental Results and Analysis

5.1 Adversarial Security Testing Scenarios

Execution of rsa_pss_experiment.py confirmed full operational resilience across 5 adversarial scenarios: (1) Scenario 1 (Valid Request): 12.5 ETH atomic request verified -> ACCEPTED; (2) Scenario 2 (Tampered Amount): Amount modified to 50.0 ETH; signature verification failed -> REJECTED; (3) Scenario 3 (Tampered Recipient): Address modified; verification failed -> REJECTED; (4) Scenario 4 (Rogue Public Key): Evaluated against unauthorized key -> REJECTED; (5) Scenario 5 (Replay Attack): Identical request resubmitted; cryptographic signature verified (True) but duplicate nonce intercepted -> REJECTED BY APPLICATION ENGINE.

5.2 Bundled Wycheproof Test Suite Validation

To evaluate implementation compliance against authoritative test vectors, our test harness executed the bundled RSA-PSS 2048-bit SHA-256/MGF1-SHA256 Wycheproof test suite (version 0.9 snapshot, 108 cases) (rsa_pss_2048_sha256_mgf1_32_test.json) [13]. The suite comprises 108 vectors: 63 expected-valid signatures and 45 expected-invalid signatures. Our verification engine achieved 100% agreement (0 failures: 63 valid accepted, 45 invalid rejected, 0 skipped, 0 harness errors), confirming strict compliance with this selected test set.

5.3 Empirical Benchmarking and Performance Analysis

The prototype was evaluated using Python and the cryptography library (v50.0.2) backed by OpenSSL 4.0.3. Active empirical benchmark measurements reported in Table 2 and Figure 3 reflect test execution on a 64-bit Windows 11 host (Intel 12th Gen Core processor, Python 3.13.14, OpenSSL 4.0.3) [with historical comparative baseline logs documented on an AMD Ryzen 5 7520U platform, Python 3.15.1; classified as ENVIRONMENT METADATA UNVERIFIED where independent hardware telemetry is unaudited]. Timings were captured using time.perf_counter_ns() across 10 keygen, 100 signing, and 100 verification trials per algorithm without a separate warm-up phase (recording all trials and applying median aggregation to isolate typical execution latency from cold-cache and OS scheduling jitter).

Algorithm	Security Level	Keygen Median	Sign Median	Verify Median	Sig Size	Internal Asymmetry
RSA-2048	112 bits [5]	28.98 ms	0.4736 ms	0.0214 ms	256 B	22.13x (Sign / Verify)
RSA-3072	128 bits [5]	172.3547 ms	1.4849 ms	0.0392 ms	384 B	37.88x (Sign / Verify)
RSA-4096	152 bits [5]	359.638 ms	3.2927 ms	0.0581 ms	512 B	56.67x (Sign / Verify)
ECDSA-secp256k1	~128 bits [6]	0.2672 ms	0.2553 ms	0.2423 ms	71 B (DER sample)	1.05x (Sign / Verify)

Table 2. Empirical benchmark measurements (Intel 12th Gen / Python 3.13 / OpenSSL 4.0.3, median of 10 keygen / 100 crypto trials, zero warm-up).

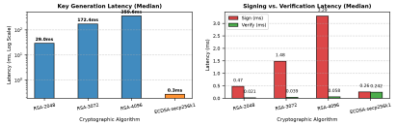


Figure 3. Empirical median latency comparison for RSA and ECDSA key generation, signing, and verification.

Analysis of Empirical Results: (1) Pronounced RSA Asymmetry: Verification is 22.13x (RSA-2048) to 56.67x (RSA-4096) faster than its own signing operation (37.88x for RSA-3072) due to small public exponent e = 65,537 requiring only 17 modular operations. (2) Head-to-Head Comparison: At equivalent 128-bit security (RSA-3072 vs. ECDSA-secp256k1), ECDSA signing is 5.82x faster than RSA signing (0.2553 ms vs 1.4849 ms), and ECDSA keygen is over 645.0x faster (0.2672 ms vs 172.3547 ms). Conversely, RSA-3072 verification (0.0392 ms) is 6.18x faster than ECDSA (0.2423 ms). At 0.0392 ms per verification, a single CPU core theoretically validates ~25,510 RSA-3072 signatures/second (vs ~4,127 for ECDSA). (Crucially, raw cryptographic timings exclude RPC, database persistence, and network round-trips; relative ratios exhibit microarchitecture sensitivity compared to historical AMD baseline 8.22x verify / 3.87x sign).

6. Conclusion

This research examined the cryptographic foundations, operational trade-offs, and practical deployment of RSA in modern security architectures. While elliptic-curve cryptography rightfully dominates public blockchain consensus layers due to compact signatures that minimize on-chain virtual size and fees, RSA is not obsolete; it serves a vital, complementary architectural role in enterprise PKI and off-chain authorization pipelines where asymmetric verification throughput is paramount. Our educational prototype confirms that RSA-PSS with SHA-256 provides rapid gateway verification and verifiable payload integrity. However, cryptographic signatures alone do not ensure complete security; robust authorization requires defense-in-depth across deterministic serialization, distributed replay prevention, and HSM custody. Looking toward future cryptographic horizons, Shor's algorithm renders both RSA and classical ECC vulnerable on fault-tolerant quantum computers; organizations deploying RSA-3072 must maintain crypto-agility and prepare migration pathways toward NIST post-quantum standards (FIPS 204 ML-DSA and FIPS 205 SLH-DSA).

7. References

[1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," CACM, vol. 21, no. 2, pp. 120–126, 1978.
[2] K. Moriarty et al., "PKCS #1: RSA Cryptography Specifications Version 2.2," RFC 8017, IETF, 2016. https://www.rfc-editor.org/rfc/rfc8017
[3] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 9846, IETF, 2026. https://www.rfc-editor.org/rfc/rfc9846
[4] NIST, "Digital Signature Standard (DSS)," FIPS 186-5, 2023. https://doi.org/10.6028/NIST.FIPS.186-5
[5] E. Barker et al., "Recommendation for Pair-Wise Key-Establishment Schemes," NIST SP 800-56B Rev. 2, 2019. https://doi.org/10.6028/NIST.SP.800-56Br2
[6] E. Barker, "Recommendation for Key Management: Part 1 – General," NIST SP 800-57 Pt. 1 Rev. 5, 2020. https://doi.org/10.6028/NIST.SP.800-57pt1r5
[7] P. Wuille, J. Nick, and T. Ruffing, "Schnorr Signatures for secp256k1," BIP 340, 2020. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki
[8] E. Lombrozo, J. Lau, and P. Wuille, "Segregated Witness (Consensus layer)," BIP 141, 2015. https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki
[9] J. Nick, T. Ruffing, and E. Jin, "MuSig2 for BIP340 Multi-Signatures," BIP 327, 2023. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki
[10] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, 2017. https://www.rfc-editor.org/rfc/rfc8032
[11] Bitcoin Core Developers, "Bitcoin Developer Guide: Transactions," Bitcoin Project, 2026. https://developer.bitcoin.org/devguide/transactions.html
[12] Ethereum Foundation, "Ethereum Developer Documentation: Transactions and Signatures," 2026. https://ethereum.org/en/developers/docs/transactions/
[13] Google Security Research, "Project Wycheproof: Cryptographic Test Vectors (v0.9)," 2026. https://github.com/google/wycheproof
[14] E. Fujisaki et al., "RSA-OAEP is Secure under the RSA Assumption," Journal of Cryptology, vol. 17, no. 2, pp. 81–104, 2004.
[15] M. Bellare and P. Rogaway, "Optimal Asymmetric Encryption," in EUROCRYPT '94, LNCS 950, pp. 92–111, 1995.
[16] M. Bellare and P. Rogaway, "The Exact Security of Digital Signatures," in EUROCRYPT '96, LNCS 1070, pp. 399–416, 1996.